

Sesión 11: Widgets de Entrada y Formularios (Form, TextFormField, Validación de Datos, FocusNode y Manejo del Teclado Nativo)

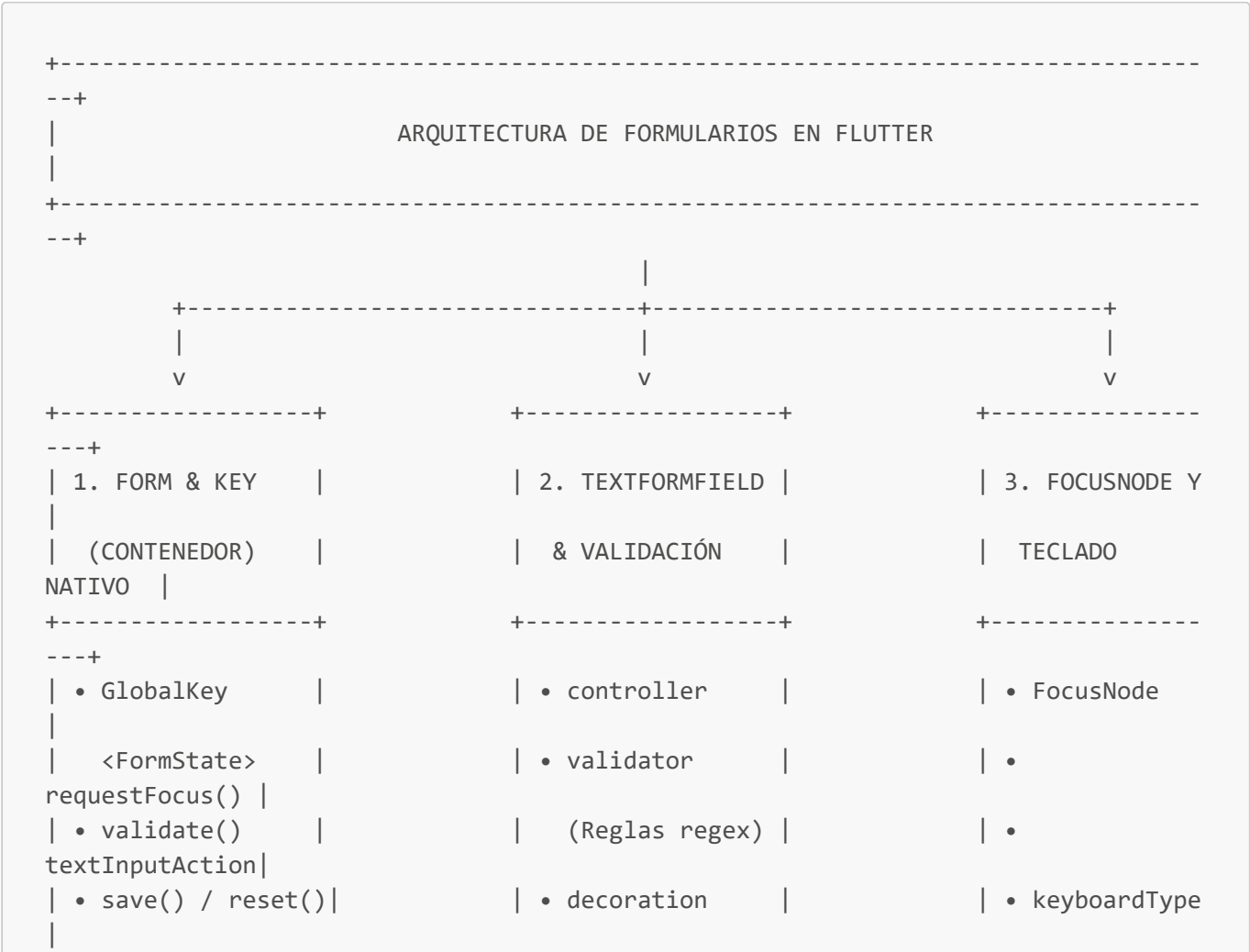
Introducción

La captura de datos e interacción directa con el usuario representa un pilar crítico en las aplicaciones móviles. Diseñar formularios accesibles y robustos exige gestionar de manera transparente la entrada de texto, validar la integridad de la información en tiempo real, controlar la navegación entre campos mediante el teclado nativo y asegurar una óptima experiencia de usuario (UX).

En esta sesión abordaremos la arquitectura de formularios en Flutter. Analizaremos el uso del contenedor **Form**, el widget especializado **TextFormField**, las reglas declarativas de validación de datos, la gestión programática del foco mediante **FocusNode** y el control del teclado nativo para optimizar el flujo de captura de información.

Conceptos y Definiciones

La gestión de formularios en Flutter articula el estado del widget, las restricciones de validación y los eventos del sistema operativo.



```
+-----+           +-----+           +-----+
---+
```

1. El Contenedor **Form** y **GlobalKey<FormState>**

- **Form**: Widget contenedor agrupador que sirve como punto centralizado de control para múltiples campos de entrada (**FormField**).
- **GlobalKey<FormState>**: Clave global que identifica de manera única el estado interno del **Form** (**FormState**) en el árbol de elementos. Permite invocar métodos de ciclo de vida del formulario desde cualquier punto del contexto:
- **currentState!.validate()**: Ejecuta secuencialmente la función **validator** de todos los campos contenidos. Devuelve **true** si todas las validaciones son exitosas o **false** si al menos un campo presenta un error.
- **currentState!.save()**: Dispara la llamada al callback **onSaved** de cada campo para extraer los valores.
- **currentState!.reset()**: Limpia y restaura todos los campos a sus valores iniciales.

2. Campos de Entrada Especializados: **TextFormField**

A diferencia del widget básico **TextField**, **TextFormField** está integrado directamente con el ecosistema del widget **Form**.

- **controller (TextEditingController)**: Controla el texto que se despliega en el campo y permite escuchar cambios en tiempo real o leer/modificar el contenido programáticamente. Debe ser destruido (**dispose()**) en el ciclo de vida del widget para evitar fugas de memoria.
- **validator**: Recibe una función de callback **String? Function(String? value)** que evalúa el texto ingresado. Debe retornar **null** si el valor es válido, o un mensaje **String** explicativo si el valor viola alguna regla de negocio.
- **autovalidateMode**: Controla cuándo se ejecuta la validación:
- **AutovalidateMode.disabled**: Solo valida al invocar explícitamente **validate()**.
- **AutovalidateMode.onUserInteraction**: Valida dinámicamente conforme el usuario edita el campo tras la primera interacción.

3. Control de Foco (**FocusNode**) e Integración con el Teclado Nativo

- **FocusNode**: Objeto que administra el foco del teclado del sistema para un widget específico.
- Permite redirigir el foco programáticamente entre campos al presionar la tecla "Siguiente" mediante **FocusScope.of(context).requestFocus(siguienteFocusNode)**.
- Debe ser liberado mediante **dispose()** al finalizar su ciclo de vida.
- **Configuración del Teclado Nativo**:
- **keyboardType**: Ajusta la distribución del teclado táctil según el tipo de dato (**TextInputType.emailAddress**, **number**, **phone**, **text**).

- **textInputAction**: Modifica la acción visual del botón de confirmación en la esquina inferior derecha del teclado (`TextInputAction.next`, `done`, `search`).
- **obscureText**: Oculta los caracteres ingresados, ideal para contraseñas o datos sensibles.

Ejemplo Completo con Código Fuente: Formulario de Registro de Usuario con Validaciones y Flujo de Foco

El siguiente código implementa una pantalla completa de registro que incluye validaciones mediante expresiones regulares, gestión secuencial del foco de teclado con `FocusNode`, ocultamiento dinámico de contraseña y manejo centralizado del estado del `Form`.

Código Fuente Explicado (`lib/main.dart`)

```
import 'package:flutter/material.dart';

void main() {
  runApp(const DemoFormulariosApp());
}

/// Widget Raíz
class DemoFormulariosApp extends StatelessWidget {
  const DemoFormulariosApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ESCOM - Entrada y Formularios',
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(
          seedColor: const Color(0xFF6B112D), // Guinda Institucional
        ),
      ),
      home: const PantallaRegistroFormulario(),
    );
  }
}

/// Pantalla Principal con Formulario
class PantallaRegistroFormulario extends StatefulWidget {
  const PantallaRegistroFormulario({super.key});

  @override
  State<PantallaRegistroFormulario> createState() =>
    _PantallaRegistroFormularioState();
}

class _PantallaRegistroFormularioState
  extends State<PantallaRegistroFormulario> {
```

```
// Clave global para identificar y controlar el estado del Formulario
final _formKey = GlobalKey<FormState>();

// Controladores para la lectura del texto
final TextEditingController _nombreController = TextEditingController();
final TextEditingController _correoController = TextEditingController();
final TextEditingController _passwordController = TextEditingController();

// Nodos de foco para administrar la navegación con el teclado nativo
final FocusNode _nombreFocus = FocusNode();
final FocusNode _correoFocus = FocusNode();
final FocusNode _passwordFocus = FocusNode();

bool _ocultarPassword = true;

@override
void dispose() {
  // Liberación estricta de recursos para evitar fugas de memoria
  _nombreController.dispose();
  _correoController.dispose();
  _passwordController.dispose();
  _nombreFocus.dispose();
  _correoFocus.dispose();
  _passwordFocus.dispose();
  super.dispose();
}

/// Método para procesar y enviar el formulario
void _procesarFormulario() {
  // Ejecuta la validación de todos los TextFormField contenidos en el Form
  if (_formKey.currentState!.validate()) {
    // Ocultar teclado nativo
    FocusScope.of(context).unfocus();

    ScaffoldMessenger.of(context).showSnackBar(
      SnackBar(
        content: Text(
          '¡Registro exitoso! Usuario: ${_nombreController.text}',
        ),
        backgroundColor: Colors.green.shade700,
      ),
    );
  }
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('Registro de Usuario - ESCOM'),
      backgroundColor: Theme.of(context).colorScheme.primary,
      foregroundColor: Colors.white,
    ),
    body: SingleChildScrollView(
```

```
padding: const EdgeInsets.all(24.0),
child: Form(
  key: _formKey,
  autovalidateMode: AutovalidateMode.onUserInteraction,
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.stretch,
    children: [
      const Text(
        'Ingresa tus credenciales de acceso',
        style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
      ),
      const SizedBox(height: 20),

      // CAMPO 1: NOMBRE COMPLETO
      TextFormField(
        controller: _nombreController,
        focusNode: _nombreFocus,
        keyboardType: TextInputType.name,
        textInputAction: TextInputAction.next,
        decoration: const InputDecoration(
          labelText: 'Nombre Completo',
          hintText: 'ej. Ozzy',
          prefixIcon: Icon(Icons.person),
          border: OutlineInputBorder(),
        ),
        onFieldSubmitted: (_) {
          // Mover el foco programáticamente al siguiente campo
          FocusScope.of(context).requestFocus(_correoFocus);
        },
        validator: (value) {
          if (value == null || value.trim().isEmpty) {
            return 'El nombre es obligatorio.';
          }
          if (value.trim().length < 3) {
            return 'Ingresa al menos 3 caracteres.';
          }
          return null;
        },
      ),
      const SizedBox(height: 16),

      // CAMPO 2: CORREO ELECTRÓNICO INSTITUCIONAL
      TextFormField(
        controller: _correoController,
        focusNode: _correoFocus,
        keyboardType: TextInputType.emailAddress,
        textInputAction: TextInputAction.next,
        decoration: const InputDecoration(
          labelText: 'Correo Electrónico',
          hintText: 'usuario@ipn.mx',
          prefixIcon: Icon(Icons.email),
          border: OutlineInputBorder(),
        ),
        onFieldSubmitted: (_) {
```

```

        FocusScope.of(context).requestFocus(_passwordFocus);
      },
      validator: (value) {
        if (value == null || value.trim().isEmpty) {
          return 'El correo es obligatorio.';
        }
        // Expresión regular para validar formato de correo
        final emailRegex =
          RegExp(r'^[\w-\.\.]+\@([\w-]+\.\.)+[\w-]{2,4}$');
        if (!emailRegex.hasMatch(value.trim())) {
          return 'Ingrese un correo electrónico válido.';
        }
        return null;
      },
    ),
    const SizedBox(height: 16),

    // CAMPO 3: CONTRASEÑA
    TextFormField(
      controller: _passwordController,
      focusNode: _passwordFocus,
      obscureText: _ocultarPassword,
      keyboardType: TextInputType.visiblePassword,
      textInputAction: TextInputAction.done,
      decoration: InputDecoration(
        labelText: 'Contraseña',
        prefixIcon: const Icon(Icons.lock),
        border: const OutlineInputBorder(),
        suffixIcon: IconButton(
          icon: Icon(
            _ocultarPassword
              ? Icons.visibility_off
              : Icons.visibility,
          ),
          onPressed: () {
            setState(() {
              _ocultarPassword = !_ocultarPassword;
            });
          },
        ),
      ),
      onFieldSubmitted: (_) {
        _procesarFormulario();
      },
      validator: (value) {
        if (value == null || value.isEmpty) {
          return 'La contraseña es obligatoria.';
        }
        if (value.length < 6) {
          return 'La contraseña debe tener al menos 6 caracteres.';
        }
        return null;
      },
    ),
  ),

```

```

        const SizedBox(height: 28),

        // BOTÓN DE ENVÍO
        ElevatedButton.icon(
          onPressed: _procesarFormulario,
          icon: const Icon(Icons.check_circle),
          label: const Text(
            'REGISTRAR USUARIO',
            style: TextStyle(fontWeight: FontWeight.bold),
          ),
          style: ElevatedButton.styleFrom(
            padding: const EdgeInsets.symmetric(vertical: 16),
            backgroundColor: Theme.of(context).colorScheme.primary,
            foregroundColor: Colors.white,
          ),
        ),
      ],
    ),
  ),
);
}
}

```

Prácticas Guiadas / Laboratorio

Práctica 1: Validación Personalizada de Coincidencia de Contraseñas

1. Agrega un cuarto campo `TextFormField` llamado "Confirmar Contraseña" con su respectivo `TextEditingController` y `FocusNode`.
2. Implementa en su callback `validator` una regla de negocio que compare la entrada actual con la propiedad `text` del `_passwordController`:

```

validator: (value) {
  if (value == null || value.isEmpty) {
    return 'Confirme su contraseña.';
  }
  if (value != _passwordController.text) {
    return 'Las contraseñas no coinciden.';
  }
  return null;
}

```

3. Configura el `textInputAction` del campo de contraseña principal a `TextInputAction.next` para redirigir el foco al nuevo campo de confirmación antes de procesar el formulario.

Práctica 2: Formateo y Restricción de Entradas mediante `inputFormatters`

1. Añade un campo para ingresar un número telefónico o un código numérico (ej. Boleta o CURP).
2. Asigna la propiedad `keyboardType: TextInputType.number`.
3. Importa `package:flutter/services.dart` e integra la propiedad `inputFormatters` para restringir la entrada a solo dígitos numéricos y limitar la longitud máxima a 10 caracteres:

```
inputFormatters: [  
  FilteringTextInputFormatter.digitsOnly,  
  LengthLimitingTextInputFormatter(10),  
],
```

Resumen y Puntos Clave

- **Contenedor `Form`:** Centraliza y simplifica la gestión, validación y reseteo de múltiples campos de entrada en la interfaz de usuario.
- **`GlobalKey<FormState>`:** Proporciona acceso directo al estado del `Form` para invocar métodos globales como `validate()`, `save()` o `reset()`.
- **`TextFormField` y `validator`:** Permite definir funciones declarativas de validación que retornan `null` ante entradas correctas o cadenas de error cuando la regla falla.
- **Manejo de Foco (`FocusNode`):** Controla el flujo secuencial de navegación entre campos al presionar la acción del teclado nativo, mejorando la usabilidad.
- **Optimización de Teclado Nativo:** Asignar correctamente `keyboardType` y `textInputAction` adapta el teclado táctil del dispositivo al contexto específico de los datos esperados.
- **Gestión de Recursos:** Es fundamental llamar a `dispose()` en controladores de texto (`TextEditingController`) y nodos de foco (`FocusNode`) dentro del ciclo de vida del widget para prevenir fugas de memoria.

Bibliografía y Referencias

1. **Google LLC.** (2024). *Build a form with validation - Flutter Tutorials*. Recuperado de: <https://docs.flutter.dev/cookbook/forms/validation>
2. **Google LLC.** (2024). *TextFormField class documentation - Flutter API*. Recuperado de: <https://api.flutter.dev/flutter/material/TextFormField-class.html>
3. **Google LLC.** (2024). *FocusNode class documentation - Flutter API*. Recuperado de: <https://api.flutter.dev/flutter/widgets/FocusNode-class.html>
4. **Moedano, B.** (2023). *Desarrollo de Aplicaciones Móviles Adaptativas con Flutter*. Editorial Alfaomega. ISBN: 978-6075389102.